

Introduction to Embedded Systems

Course Code: 244066

Lab 2

Cortex-M3 Exceptions and STM32 Interrupts

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Instructions:

- Complete every task in the lab practice session.
- For each task, include a 2–5 line description along with a screenshot of both your code and its output.
- Submit your lab report by email to Imran.Nadeem@outlook.com, using the subject line format as:
- **ES24-[lab number]-[registration number]**.
- Lab report need to be submitted before **11:59 pm** tonight.
- Submissions received after the deadline will not be accepted.

Overview

Overview of Learning Outcomes

By completing these simulated labs, you will:

- Understand **difference between interrupts and exceptions**.
- Be able to configure and test **NVIC priorities and EXTI lines**.
- Observe **ISR behavior** and **context switching** in simulation.
- Practice **debugging fault exceptions** using STM32CubeIDE.
- Gain confidence to apply interrupt-driven design in real hardware later.

Lab 1: GPIO External Interrupt (EXTI) Simulation

To understand how a GPIO pin event (such as a button press) generates an interrupt and how the ISR responds.

Steps

1. Create a new STM32 project.
2. Configure **PA0** as input and connect it to **EXTI Line 0**.
3. Configure EXTI for **rising edge trigger**.
4. Write an ISR (EXTI0_IRQHandler) to toggle a software variable led_state.
5. In debug mode, simulate the signal change using the **watch window** or by manually writing to EXTI->PR.
6. Observe changes in led_state each time the interrupt is triggered.

Lab 2: NVIC Priority and Preemption Simulation

To demonstrate interrupt priority and preemption using software-generated interrupts.

Steps

1.Create two dummy ISRs: Timer_IRQ_Handler() and USART_IRQ_Handler().

2.Set Timer_IRQ with **higher priority** and USART_IRQ with **lower priority** in NVIC.

3.Trigger both using software:

```
NVIC_SetPendingIRQ(USART_IRQn);
NVIC_SetPendingIRQ(TIM2_IRQn);
```

4. Observe the execution order in the **debug call stack**.

5. Swap priorities and repeat to see the effect of preemption.

Lab 3: Timer Interrupt – Virtual LED Blinking

To generate periodic interrupts using a timer and simulate LED blinking behavior.

Steps

1. Configure **Timer2** to generate interrupts every 500ms.
2. In the ISR (TIM2_IRQHandler), toggle a variable led_flag.
3. Run the simulation and observe led_flag in the **Expressions** window.
4. Adjust the prescaler and auto-reload values to change blinking speed.

Lab 4: UART Interrupt Simulation (Echo Program)

To simulate UART-based interrupt handling using a virtual console.

Steps

1. Initialize USART2 for communication (9600 baud).
2. Enable the RX interrupt (USART_IT_RXNE).
3. In USART2_IRQHandler, read the received byte and echo it back using HAL_UART_Transmit().
4. Send characters via the simulation console.
5. Observe how the ISR executes for each received byte.

Lab 5: Fault Exception Simulation

To observe how Cortex-M3 handles fault exceptions such as HardFault and UsageFault.

Steps

1. Write the following code snippets one by one:

Usage Fault:

```
int x = 1 / 0;
```

Bus Fault:

```
*(volatile int*)0xFFFFFFF0 = 5;
```

2. Observe which handler executes (HardFault_Handler, UsageFault_Handler, etc.).
3. Place breakpoints in the fault handlers and inspect the stack frame.